

INTERNAL REFERENCE · UPDATED AFTER THE LELLON BUILD

The stack we keep reaching for

Lellon is the first app built on this machine, not the last. This is everything about *how* it's built — the platform, the backend, the architectural patterns, the testing conventions — that has nothing to do with medications and everything to do with the next app. **Skip anything Lellon-specific** (dose schedules, pill imprints, caregiver sharing); this is the reusable substrate underneath it.

At a glance

- Platform & project
- Firebase backend
- Local storage & sync
- Extensions & background
- Notifications
- Testing conventions
- Other integrations
- This machine, specifically
- Starting the next app

CATEGORY	WHAT WE USED	HOW IT SHOWS UP IN CODE
UI	SwiftUI, multiplatform target	<code>@Observable</code> services injected via <code>.environment()</code>
Local storage	SwiftData	<code>@Model</code> , <code>ModelContainer</code> , <code>ModelContext</code>

CATEGORY	WHAT WE USED	HOW IT SHOWS UP IN CODE
Auth	Firebase Auth + Sign in with Apple + Google Sign-In	<code>FirebaseAuth</code> , <code>AuthenticationServices</code> , <code>GoogleSignIn</code>
Cloud database	Cloud Firestore	<code>FirebaseFirestore</code> , <code>firestore.rules</code>
Server logic	Cloud Functions (2nd gen, Node.js)	<code>firebase-functions/v2</code> , Firestore-trigger pattern
Push	Firebase Cloud Messaging	<code>FirebaseMessaging</code> , <code>admin.messaging()</code>
Local notifications	UserNotifications framework	<code>UNUserNotificationCenter</code> , custom categories/actions
Widgets / Live Activity	WidgetKit + ActivityKit	App Group + <code>AppIntents</code>
Background execution	BGTaskScheduler	<code>BGAppRefreshTask</code>
Testing	Swift Testing (not XCTest)	<code>@Test</code> , <code>@Suite</code> , <code>#expect</code>
Transactional email	SendGrid, server-side only	called from a Cloud Function, never the client
Hashing	CryptoKit	<code>SHA256</code> for a local, no-server-round-trip gate

PLATFORM

Platform & project setup

The baseline every app on this machine can probably start from as-is.

SwiftUI declarative UI, Swift 5	Every screen is a SwiftUI view; no UIKit view controllers except where a system API demands one (see below). The app target is built multiplatform-capable (iOS/macOS/visionOS) from day one, even when only iOS actually ships — costs nothing up front and keeps the door open.
Info.plist strategy GENERATE_INFOPLIST_FILE + a small merge file	Xcode's auto-generated Info.plist handles the routine keys (display name, orientations, scene manifest). A small, separately-committed <code>Info.plist</code> is merged in on top of it (via the <code>INFOPLIST_FILE</code> build setting) for everything auto-generation doesn't cover — usage-

description strings, custom URL schemes, `UIBackgroundModes`, `BGTaskSchedulerPermittedIdentifiers`. No giant hand-maintained plist, but still fully customizable.

System frameworks

`UIKit`,
`BackgroundTasks`,
`WidgetKit`, `AppIntents`

None of these needed an explicit entry in the Xcode "Frameworks" build phase — a plain `import` was enough for modern Xcode to auto-link them. Don't reach for a manual pbxproj edit for a system framework until an actual link error says otherwise.

BACKEND

Firestore, and how we used it

Pulled in once via Swift Package Manager (the `firebase-ios-sdk` package). Only four of its products are actually linked into the app target — resist adding more until something genuinely needs it.

Firestore Auth

`FirestoreAuth`

Email/password, Sign in with Apple, and Google Sign-In all land in the same `FirestoreAuth` user. A fully local **guest mode** — no network call, nothing persisted, every launch starts fresh — is worth building in from the start as a first-class option, not an afterthought; it's what let every other feature stay opt-in rather than account-gated.

Firestore Firestore

`FirestoreFirestore`

The cloud database. `Firestore.rules` is the real authorization layer, not a formality — every collection's access is enforced there, independent of (and never trusting) anything the client claims. Two rules techniques worth carrying forward: reusable `function` helpers for a grant check used in more than one place, and validating a field's *allowed transition* (e.g. only one participant may flip a status from A to B), not just which fields a write is allowed to touch.

Firestore Functions

`firebase-functions v2`,
`Node.js`

2nd-gen functions only: `onDocumentCreated` for "the client wrote a request document, something server-side needs to react" (sending an email, sending a push, anything needing a secret the client must never see), and `onSchedule` for periodic polling jobs. **Deliberately never added the FirestoreFunctions client SDK** for callable functions — the create-a-Firestore-doc-and-let-a-trigger-react shape covers everything we've needed so far, with one fewer dependency and one fewer thing to link.

Firestore Messaging

`FirestoreMessaging`

Device push tokens are captured client-side and stored per-user in Firestore (behind rules that let only that user and the Admin SDK read

it); a Cloud Function sends the actual push via `admin.messaging().send()`. The client never sends a push directly to another user.

Deploy reality

manual, for now

No Firebase CLI project link exists in this environment yet — `firestore.rules` changes are hand-pasted into the Console, and Cloud Functions are deployed manually rather than from a CI pipeline. Worth setting up a real `firebase` CLI link on the next project if a deploy pipeline matters from the outset.

ARCHITECTURE

Local storage & the sync pattern

The most reusable thing here isn't a package — it's the shape of the offline-first, opt-in-cloud architecture itself.

SwiftData

not Core Data directly,
not Realm

Every model is an `@Model` class; the app owns one `ModelContainer`, and views/services work through `ModelContext`. It's the default choice for on-device storage on this stack going forward.

On-device-first, cloud opt-in

the "StorageMode" pattern

A simple two-case mode (on-device-only vs. cloud-synced) that everything else checks before ever touching the network — a sync service is a total no-op unless the user has actively opted in *and* is signed in. This is what let cloud sync, sharing, and cross-device features get added later without changing how the app behaves for someone who never opts in.

Conflict resolution

last-write-wins by
timestamp

Every syncable record carries its own `updatedAt`; reconciliation compares local vs. remote and takes whichever is newer. Deliberately not true multi-writer merge — a documented simplification that's been sufficient so far, not an oversight.

Tombstones for deletes

a dedicated "deletedIds" collection

A delete writes a small tombstone record *before* attempting the real delete — so if the app dies mid-request, the next sync still knows never to resurrect that id, and retries the real delete itself. Skipping this step is exactly how a deleted record quietly comes back from the dead on the next sync.

Retry queue for failed pushes

a small UserDefaults-backed queue

A network write that fails shouldn't just vanish silently — queue it (keyed by record id, de-duped, latest wins), and retry opportunistically

on the next foreground or background-task run. Cheap to build, and it closes an entire class of "my change never made it to the cloud" bugs.

Owner-key scoping

an explicit field, not an assumption

Every locally-stored row carries its own owner identifier. On a shared device, this is what stops one account's leftover local data from bleeding into a different account signed in afterward — never assume "the local store only ever has one account's data in it."

EXTENSIONS

Widgets, Live Activities & background work

The pattern that makes all three tractable: an extension process can't touch the main app's `SwiftData` store directly, so it never tries to.

App Group

the shared boundary

A widget or Live Activity writes into (and reads from) an App Group container — a lightweight, read-only snapshot for display, and a small pending-action queue for anything interactive ("mark taken"). The main app is the only thing that ever actually mutates real data; it drains that queue on its own next launch/foreground/background-run.

WidgetKit + ActivityKit

Home Screen widgets, Lock Screen, Dynamic Island

Interactive buttons on either go through `AppIntents`, not a deep link. `WidgetCenter.shared.reloadTimelines(ofKind:)` after any relevant local mutation is what keeps a widget from just showing stale data until the next OS-scheduled refresh.

BGTaskScheduler

`BGAppRefreshTask`

For work that should happen even if the app is never reopened. Needs `BGTaskSchedulerPermittedIdentifiers` in `Info.plist`, registration before `application(_:didFinishLaunchingWithOptions:)` returns, and (for it to actually get real scheduling priority from the system) `fetch` in `UIBackgroundModes`. It's opportunistic, never on-demand — expect it to run "eventually," not "on time."

NOTIFICATIONS

Local & remote notifications

UNUserNotificationCenter

local, scheduled notifications

Custom `UNNotificationCategory` / `UNNotificationAction` pairs give a notification its own interactive buttons. Anything that needs to resolve silently in the background should queue itself (see the retry-queue pattern above) rather than assume it can touch app state

directly from the notification-response callback — that callback can fire on a cold launch before anything else exists yet.

UIApplicationDelegateAdaptor

the UIKit bridge SwiftUI still needs

Remote-notification device-token registration and notification-response handling are still UIKit-only callbacks with no SwiftUI-native equivalent. One small `UIApplicationDelegate` conformer, wired in via `@UIApplicationDelegateAdaptor`, is the one piece of UIKit most apps on this stack will still need.

Time-sensitive interruption level

requires its own entitlement

`content.interruptionLevel = .timeSensitive` breaks through Focus modes — but silently degrades to a normal banner without the Time Sensitive Notifications entitlement. Safe to set either way; just don't expect the behavior without also adding the entitlement.

TESTING

Testing conventions

Swift Testing

not XCTest, for new Swift unit tests

`import Testing, @Test, @Suite, #expect / #require`. XCTest is still what UI test targets require (Swift Testing doesn't run in that context), so the two coexist rather than one replacing the other outright.

The seam pattern

protocol + real adapter + fake

The single most-reused architectural idea on this project: every service that talks to Firestore/notifications/the network sits behind a small protocol (`XStoring`), with a real adapter (`FirestoreXAdapter`) as its default implementation and a plain in-memory fake for tests. Nothing has ever needed a real network connection or a mocking framework to get real unit-test coverage.

Isolated UserDefaults per test

never `.standard`

`UserDefaults(suiteName: "SomeTest-\(UUID())")` instead of the shared default store — keeps tests safe to run in parallel, with no shared, order-dependent state between them.

node:test for Cloud Functions

no Jest, no Mocha

Node's own built-in test runner (`node:test` + `node:assert/strict`) is enough for Cloud Function unit tests, run against the Firebase emulator suite — one less dependency to install per project.

Environment gap: the Firestore emulator needs a local Java runtime, which isn't installed on this machine yet — so the emulator-backed suites (`firestore.rules` tests, the fuller Cloud Function integration tests) can be written and syntax-checked (`node --check`) here, but not actually run

until Java is installed. Worth fixing once, on this machine, for every future project rather than rediscovering it each time.

INTEGRATIONS

Other integrations worth remembering

HealthKit

`clinical record import`

Read-only clinical-record access, parsed into the app's own local model — never written back to Health.

MapKit + CLGeocoder

`place lookup`

Nearby-place search and address ↔ coordinate resolution, both fully on-device/Apple-hosted — no third-party maps dependency needed so far.

CryptoKit

`SHA256, for a local-only gate`

When a feature needs to be "gated by a shared code" but doesn't need real access control (it only ever affects the same user's own data), hashing the code and comparing hashes — never committing the plaintext to source — is a fine, server-round-trip-free pattern. Not a substitute for real authorization when the stakes are higher than that.

SendGrid, server-side only

`via a Cloud Function, never the client`

Transactional email (an issue report, an alert) is sent from a Cloud Function using a secret held only there (`firebase functions:secrets:set`) — the client only ever writes a plain Firestore document; it never sees an API key or a recipient address. Requires one manual, external step per sender address: SendGrid's Single Sender (or domain) Verification.

Bundled offline reference data

`SQLite + FTS5 trigram search`

For a large, mostly-static structured dataset (a product catalog, a reference table, anything from a public bulk-data source): build it once, offline, into a compact `.sqlite` file with an FTS5 trigram index for fuzzy search, and ship or download it as a plain resource. Queries need no network at all. Worth the ingestion-script investment any time an app needs to search something bigger than "fits in memory as JSON."

WORKFLOW

This machine, specifically

Not framework choices — real, current limitations of this development environment, worth knowing up front on the next project rather than re-discovering mid-build.

Known gaps, as of the Lellon build:

- No Firebase CLI project link — `firestore.rules` and Cloud Functions are deployed by hand, not from a pipeline.
 - No local Java runtime — blocks the Firestore emulator suite (`firebase emulators:exec`); emulator-dependent tests can only be written and syntax-checked here, not run.
 - No reliable UI-automation tool (no `idb` ; XCTest has been flaky) — real UI verification has instead used a temporary, clearly-labeled, launch-argument-gated diagnostic view plus `xcrun simctl launch` + a screenshot, removed again once confirmed.
-
- Standing habit: after any build that succeeds, do a fresh install + relaunch on the iPhone 17 Pro simulator via `simctl` as a basic sanity pass.
 - Bundle identifier observed on this project: `VibeApp.<AppName>` — a per-project decision, not something to assume carries over automatically.

NEXT TIME

Starting the next app

A rough starting order, based on what actually got used, in the order it tends to get needed:

1. New SwiftUI app target, multiplatform-capable, plus the small merged `Info.plist` for anything auto-generation doesn't cover.
2. Firebase project + SPM package; link only
`Firestore` / `FirestoreAuth` / `FirestoreFirestore` to start — add `FirestoreMessaging` only once push actually matters.
3. SwiftData models with an explicit owner-key field from day one, even before multiple accounts are a real concern — much cheaper to have it early than to retrofit.
4. A `StorageMode` -shaped on/off for cloud sync, so every account can start fully local with zero network dependency.
5. The seam pattern (protocol + real adapter + fake) for every service from the very first one — retrofitting it later means rewriting tests that were never possible to write.
6. `firestore.rules` written and rules-tested before the first collection ships, not after.
7. Only reach for Cloud Functions, widgets, Live Activities, or background tasks once a real feature actually needs one — none of them are free complexity.

Compiled from the Lellon codebase — a snapshot of decisions made building that app, kept here because the next app will face the same first hundred questions.

What Cookbook added to the reusable stack

The Lellon-era reference above still holds. These are the additional, genuinely cross-project lessons — mostly Firebase gotchas that cost real debugging time — learned building Cookbook (Personal Cookbook Cloud Sync, specifically) that the next app should start from rather than rediscover.

BACKEND

Firestore list-query rules need every filter present in the query itself

cost a full debugging session on Cookbook

A security rule that checks `resource.data.ownerUserID` on a list query only matches requests that themselves include a `.whereField("ownerUserID", isEqualTo:)` (or equivalent) filter — the rule cannot retroactively narrow an unfiltered query, so an unfiltered list request that a broader rule would otherwise have allowed is instead denied outright. If a list query is denied with "Missing or insufficient permissions" and the equivalent single-document read works fine, check the query's own filters before suspecting the rule logic.

Cloud Storage security rules require the Storage SDK, not a raw URLSession

silently broke photo sync until traced with logging

A Storage rule of `request.auth != null` is never satisfied by a plain, unauthenticated `URLSession` request against a `storage.googleapis.com` download URL, even one obtained from `Storage.storage().reference(forURL:).downloadURL()`. Route any authenticated Storage download through the SDK itself — `Storage.storage().reference(forURL:).data(maxSize:)` — not a raw HTTP client.

Firebase's Spark (free) plan does not support Cloud Storage at all

a separate limitation from Firestore's own Blaze requirement — easy to conflate

As of a late-2024 Firebase policy change, Cloud Storage requires the Blaze plan outright, independent of and in addition to Firestore's own "multiple databases needs Blaze" limitation. A project can be on Blaze for Firestore reasons and still have every Storage call fail until Storage itself is confirmed enabled. Check this explicitly, in the Firebase Console, before debugging Storage failures as a code issue.

ARCHITECTURE

Downscale user photos on save, not on upload

PhotoStore.save(_:) — applies even to fully local, never-synced content

Every photo saved through the app's own photo-storage layer is resized to a fixed longest-edge cap (1600px) and re-encoded at a fixed JPEG quality (0.7) at save time, before it ever reaches disk or a sync queue. Doing this once at the single save chokepoint — rather than at each upload call site — means local storage, sync bandwidth, and cloud storage cost are all bounded for free, even for a user who never opts into cloud sync at all.

Orientation-aware layout via GeometryReader, not a fixed per-idiom asset

iPad cooking-mode background art

Where an iPad layout genuinely differs by orientation (not just size class), reading live frame geometry (`GeometryReader`, comparing width vs. height) and switching immediately on rotation reads better than a single fixed-orientation asset per idiom — worth the extra asset variants when the content is meant to fill the screen.

Compiled from the Cookbook codebase, appended to the Lellon-era reference above because the lessons generalize past either app.